

System Design Document: Orchestration, Hardening, and Security Evaluation

1. Introduction and Scope

This document outlines the design and architectural decisions for the Orchestration, Hardening, and Security Evaluation subsystem of the Accessible Multi-Factor Authentication (MFA) project. The primary objective of the overall project is to develop a highly secure authentication system tailored specifically for visually impaired users. This system must rely entirely on screen readers, keyboard navigation, and audio cues, completely removing the dependency on visual only elements such as QR codes or standard CAPTCHAs.

Within this architecture, the orchestration subsystem serves as the central coordinating authority (the "brain") of the authentication flow. It is responsible for managing the user's progress through multiple authentication factors (Password, One-Time Passwords, and WebAuthn), ensuring that these steps are completed in the correct, strict sequence. Furthermore, this subsystem is responsible for hardening the backend environment against common web vulnerabilities and implementing protocol-level defenses against sophisticated authentication attacks.

The backend infrastructure for this subsystem is designed using Python and the FastAPI framework, ensuring high performance and strong typing for security-critical endpoints.

2. Authentication State Machine (The Orchestrator)

2.1 Overview of the State Machine

In a traditional login system, authentication factors often operate independently. However, to maintain strict security and accessibility, this design implements a centralized Authentication State Machine. Instead of the client application deciding which screen to show next, the server-side Orchestrator dictates the flow. Every time a user successfully completes a login factor, the specific factor subsystem (e.g., the OTP validator) reports the success back to the Orchestrator. The Orchestrator then updates the user's progress and determines the next required action.

2.2 Design Decision and Justification

The decision to use a server-side state machine rather than client-side routing is rooted in security best practices. If the client application controls the flow, a malicious actor could intercept the network traffic and force the application to call the final login success endpoint, entirely bypassing the OTP or WebAuthn steps.

By enforcing the state on the server, the system explicitly prevents "Skipping Factor" attacks. The server verifies that the first factor is completed before allowing access to the second factor verification endpoints. This design directly aligns with the Open Worldwide Application Security Project (OWASP) Authentication guidelines, which state that multi-step authentication mechanisms must verify the successful completion of all preceding steps at every stage.

2.3 Accessible API Communication Contract

To ensure the frontend client can correctly and clearly read the current state to a visually impaired user, the Orchestrator communicates through a standardized API endpoint. This endpoint returns a structured data payload that provides the client with everything it needs without requiring the client to interpret or guess the state.

The structure of this payload includes the following critical elements, each designed with specific accessibility and security justifications:

State and Next Action: These fields clearly define the current position in the login flow (e.g., waiting for Factor 2) and the exact next step the client must take. This prevents the client and server states from drifting out of sync.

Ready-to-Speak Messages: Instead of sending error codes that the client must translate, the server sends a fully formed, plain-English sentence. This ensures that error messages and prompts make sense when spoken aloud without visual context.

Event ID Counter: This is a sequential number that increases every time the user's state changes. The client checks this number to ensure it only reads a new message when the ID changes. This directly prevents the screen reader from redundantly repeating the same instruction, which aligns with ARIA guidelines against screen reader spam.

Attempts Remaining: Visually, failed login attempts are often shown using a red border or a shaking animation. Since visually impaired users cannot see this, the server explicitly sends the exact number of attempts remaining so the screen reader can announce it clearly as a warning before the account is locked.

Expiration Timers: The payload includes the exact time the current login step will expire. This is required to meet the Web Content Accessibility Guidelines (WCAG) 2.2.1 regarding "Timing Adjustable" interfaces. It allows the frontend to announce the remaining time and offer the user an opportunity to extend their session before they are silently kicked back to the beginning.

3. Session Management and Data Storage

3.1 Redis Session Store

To maintain the authentication state between the various login steps, the system utilizes a Redis database. When a user initiates the login process, the Orchestrator generates a unique, opaque session identifier. This identifier is sent to the user's browser as a secure, HTTP-only cookie, while the actual sensitive state data is stored securely in the Redis database on the server.

3.2 Design Decision and Justification

The design decision to use Redis instead of stateless JSON Web Tokens (JWT) is driven by the need for server-side, revocable state.

During a multi-factor login process, the state is highly mutable. The system must track which factors have passed, the number of failed attempts, and the current step position. If a stateless JWT cookie were used, the server would have to issue a brand new cookie every time the state changed. More importantly, stateless cookies cannot be easily invalidated before they naturally expire. If a user fails a login step or times out, the system must instantly destroy the session to prevent an attacker from finding the cookie and continuing the login. Redis allows the Orchestrator to instantly delete or modify the session record at any time.

4. Threat Mitigation: Protocol-Level Defenses

4.1 Anti-Replay Protection

A replay attack occurs when an attacker intercepts a valid network request (such as a successfully submitted OTP code) and attempts to send it to the server a second time to gain unauthorized access.

To prevent this, the Orchestrator binds every login session to a cryptographic, single-use "nonce" (a random number used only once) or a challenge string. When the user submits their WebAuthn credential or OTP, the system compares the challenge stored in the Redis session with the challenge provided by the user.

4.2 Atomic Operations and Interleaving Protection

An interleaving attack involves an attacker starting two separate login sessions and mixing the messages between them to confuse the server and escalate their privileges.

To mitigate both interleaving and replay attacks, the system leverages atomic operations within the Redis database. When a validation endpoint (like the WebAuthn verification) looks up the session in Redis, it compares the challenge and immediately deletes the record in a single, atomic database transaction.

Because the deletion happens instantly and atomically, it is physically impossible for two concurrent requests to both succeed using the same session data. Once the challenge is consumed, the server-side state is destroyed, guaranteeing that intercepted messages can never be replayed.

4.3 Strict Time-To-Live (TTL) Matching

To ensure a smooth user experience without compromising security, the system synchronizes the expiration timers (Time-To-Live) across different components. The expiration time for a WebAuthn ceremony is configured to be strictly shorter than or equal to the main Orchestrator session expiration.

This specific design decision prevents an accessibility-unfriendly failure mode where a user might spend a long time navigating the WebAuthn prompt with their screen reader, successfully complete it, but then be rejected by the server because the underlying Orchestrator session timed out a few seconds earlier.

5. Security Hardening Mechanisms

5.1 Rate Limiting and Account Lockout

To defend against brute-force attacks (where an attacker guesses thousands of passwords a second) and credential stuffing (where an attacker uses leaked passwords from other websites), the system includes a dedicated rate-limiting middleware.

This middleware intercepts all incoming requests to the authentication endpoints. It tracks the number of failed attempts associated with both the user account and the originating IP address. If the failure threshold is crossed (for example, five incorrect attempts), the system triggers a temporary lockout.

During this time, the Orchestrator updates the state to "locked out" and provides the client with the exact wait time remaining.

5.2 Justification for Temporary Lockouts

The decision to implement temporary, exponential lockouts (e.g., a 1-minute lockout, followed by a 14-minute lockout for subsequent failures) rather than permanent account lockouts is based on guidelines from the National Institute of Standards and Technology (NIST SP 800-63B). Permanent lockouts are highly vulnerable to Denial of Service (DoS) attacks, where a malicious actor intentionally fails logins on a victim's account just to lock the legitimate user out of the system. Temporary lockouts successfully slow down automated brute-force tools while ensuring legitimate users eventually regain access.

5.3 HTTP Security Headers and Session Protection

The backend framework is configured to enforce baseline web security standards by injecting strict HTTP security headers into every response.

Strict-Transport-Security (HSTS): Ensures that the browser will only ever connect to the server over a secure, encrypted HTTPS connection, preventing attackers from downgrading the connection to intercept passwords.

Content-Security-Policy (CSP): Restricts where the browser is allowed to load resources from, which acts as a major defense against Cross-Site Scripting (XSS) attacks.

X-Frame-Options: Prevents the application from being embedded inside a hidden frame on a malicious website (Clickjacking).

Additionally, the system implements strong defenses against Cross-Site Request Forgery (CSRF). CSRF occurs when a malicious website tricks a user's browser into performing an unwanted action on the authentication system. The design mitigates this by enforcing the `SameSite=Strict` attribute on all session cookies, ensuring the browser refuses to send the user's session data if the request originates from a different website.

6. Threat Modeling and Security Evaluation

As part of the security evaluation lifecycle, the design incorporates a formal Threat Model utilizing the STRIDE methodology. This ensures all architectural decisions can be traced back to a specific, mitigated threat.

Spoofing: Mitigated by the requirement of strong, multi-factor authentication (including WebAuthn passkeys which are immune to phishing).

Tampering: Mitigated by the enforcement of HTTPS (data in transit cannot be modified) and the atomic consumption of server-side challenges (messages cannot be altered and reused).

Repudiation: Mitigated by the implementation of a comprehensive Audit Log within the Orchestrator, which records the success and failure of every login factor attempt.

Information Disclosure: Mitigated by the use of opaque session identifiers. No sensitive state data is ever exposed to the client; it is strictly maintained in the Redis database.

Denial of Service (DoS): Mitigated by the rate-limiting middleware and the strategic use of temporary, rather than permanent, account lockouts.

Elevation of Privilege: Mitigated by the strict Authentication State Machine, which absolutely prevents users from accessing protected endpoints without verifying every prerequisite factor in the correct sequence.

7. References

1. Open Worldwide Application Security Project (OWASP). Authentication Cheat Sheet. Retrieved from: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
2. National Institute of Standards and Technology (NIST). NIST Special Publication 800-63B: Digital Identity Guidelines. Retrieved from: <https://pages.nist.gov/800-63-3/sp800-63b.html>
3. World Wide Web Consortium (W3C). Web Content Accessibility Guidelines (WCAG) 2.2. Retrieved from: <https://www.w3.org/TR/WCAG22/>
4. World Wide Web Consortium (W3C). WAI-ARIA Authoring Practices Guide (APG). Retrieved from: <https://www.w3.org/WAI/ARIA/apg/>
5. Open Worldwide Application Security Project (OWASP). Top 10 Web Application Security Risks. Retrieved from: <https://owasp.org/www-project-top-ten/>
6. World Wide Web Consortium (W3C). Web Authentication: An API for accessing Public Key Credentials Level 2. Retrieved from: <https://www.w3.org/TR/webauthn-2/>